TALLINN UNIVERSITY OF TECHNOLOGY
School of Information Technologies
Study Centre for Computer Systems

Dmytro Hilei 256030MVEB

Birthday Reminder Program for Estonian Residents

Homework I

Supervisor: Risto Heinsar
Lecturer

Tallinn 2026

Author's Declaration of Originality

I hereby certify that I am the sole author of this Homework I. All the used materials, references to the literature and the work of others have been referred to. This Homework I has not been presented for examination anywhere else.

Author: Dmytro Hilei

Date: 09.03.2026

Contents

1 Task description **6**
1.1 Task variant 6

2 Data File Structure **7**
2.1 Record Structure 7
2.2 Data Fields 7
2.3 Line Length Constraint 7
2.4 Estonian ID Code Structure 7

3 Solution description **8**
3.1 Program Workflow 9
3.2 Special situations analysis 10

4 Summary **10**

References **11**

5 Appendix 1 - extra tast **11**
5.1 Command Line Arguments 11
5.2 Configuration File 11
5.3 Settings Struct 12

6 Appendix - 2 screenshots **12**
6.1 Normal program execution 12
6.2 No upcoming birthdays 12
6.3 Manual input file selection 13
6.4 No writing to output file 13

List of Figures

1 Normal program execution 12
2 Program output when no birthdays are found 12
3 Manual input file selection 13
4 Message when I choosed not to write to output file 13

List of Tables

1 Data file fields 7
2 Estonian ID code structure 8
3 Error and special situation handling 10
4 Supported command line arguments 11

1 Task description

The purpose of this task is to develop a C program that reads Estonian personal identification codes from a text file, extracts the date of birth encoded within each ID, and identifies residents whose birthday falls within the next 14 days. The program highlights individuals celebrating a milestone birthday (a multiple of 10) and marks any birthday occurring on the current date. Results are displayed in the terminal and saved to an output file.

1.1 Task variant

1. The user will be asked to enter the date (format dd.mm.yyyy).
2. The program finds and displays all people who have a birthday in the next 14 days. Display name, date of birth and how old the person will be.
3. The program will separately congratulate those who have a birthday on the entered date.
4. Additionally, those who have a milestone birthday are marked in the output.

Data file

- First name
- Last name
- Estonian personal identification number (eID, isikukood)

2 Data File Structure

2.1 Record Structure

Each record occupies one line. Fields are separated by a semicolon (`;`) and records are separated by a newline character.
Format: `FirstName;LastName;eID`
Example: `Mart;Tamm;50402126541`

2.2 Data Fields

Field	Type	Example	Description	Limits
First name	string	Mart	Person's first name	max 31 characters (buffer size 32)
Last name	string	Tamm	Person's last name	max 31 characters (buffer size 32)
eID	string	50402126541	Estonian personal identification code	11 digits (buffer size 16)

Table 1: Data file fields

2.3 Line Length Constraint

Each line in the file is limited by the program buffer size:

- Maximum line length: 127 characters
- Buffer size in program: 128 characters

2.4 Estonian ID Code Structure

The Estonian personal identification code (*isikukood*) is an 11-digit number with the following structure:

Digits	Meaning	Example (50402126541)
1	Gender and century (1–2: male/female born in 1800–1899, 3–4: 1900–1999, 5–6: 2000–2099)	5 – male, born in 2000s
2–3	Year of birth (last two digits)	04 – year 2004
4–5	Month of birth	02 – February
6–7	Day of birth	12 – 12th day
8–10	Serial number (birth order number on that day)	654
11	Checksum digit used for validation	1

Table 2: Estonian ID code structure

3 Solution description

The program parses an input file of Estonian residents, converts each 11-digit personal ID code into a date of birth, and filters residents whose birthday falls within the next 14 days of a user-provided date. Residents celebrating a milestone age (a multiple of 10) are marked with `[MILESTONE]`, and those with a birthday on the entered date are marked with `<-- TODAY!`.

Input:

- Input file path — provided via command line argument or loaded from the configuration file
- Current date — entered interactively by the user in `dd.mm.yyyy` format; invalid input is rejected and re-prompted until a valid date is received

Output:

- Matching residents are printed to the terminal and saved to an output file simultaneously, in the following format:

```
FirstName LastName - DD.MM.YYYY (turns X)  [MILESTONE]  <-- TODAY!
```

If no upcoming birthdays are found, the user is notified explicitly with a message in both the terminal and the output file.

Data Representation

Each resident record is stored in memory using a structure that contains three fields: first name, last name and the Estonian personal identification code.

Strings are stored using fixed-size character arrays to simplify parsing and avoid dynamic memory allocation.

```
struct Person {
    char firstName[MAX_NAME];
    char lastName[MAX_SURNAME];
    char eID[MAX_ID];
};
```

The program reads each line from the input file into a temporary buffer (MAX_LINE_SIZE) and then splits the line using the semicolon separator. The extracted values are copied into the structure fields.

Program Structure

The program is divided into several .c files in order to separate responsibilities and simplify debugging.

- `FilesHandaling.c` – responsible for reading the input file and loads resident records
- `ID_Analysis.c` – extracts the date of birth from the Estonian ID code and checks for the valid data
- `Output.c` – prints all valid birthdays to terminal and output file (if enabled)
- `Settings.c` – Set settings for custom way for outputing of data from configure file or from terminal if typed during compilation
- `main.c` – combines everything together

This modular design improves code readability and allows individual components of the program to be tested independently.

3.1 Program Workflow

The program executes the following steps:

1. Read resident records from the input file
2. Extract birth date information from each ID code
3. Compare the birthday with the user-provided date
4. Detect milestone birthdays (multiples of 10)
5. Print and save matching records

3.2 Special situations analysis

Issue	Description	Solution	Status
Input file not found	File path is invalid or file does not exist	Error printed to stderr, program exits with `NO_FILE` code	Solved
Output file not found	Output file cannot be opened for writing	Error printed to stderr, program exits with `NO_FILE` code	Solved
Invalid date format	User enters date in wrong format (not dd.mm.yyyy)	Error printed to stderr, input buffer cleared, user re-prompted	Solved
Invalid date values	Month or day out of valid range	Error printed to stderr, user re-prompted until valid date entered	Solved
Invalid ID length	ID code is not exactly 11 digits	Record skipped, error printed to stderr	Solved
Non-digit in ID code	ID code contains non-numeric characters	Record skipped, error printed to stderr	Solved
Invalid ID date values	Day, month or year extracted from ID is out of range	Record skipped, error printed to stderr	Solved
Config file missing	`config.conf` not found at startup	Default values used, warning printed to stderr	Solved
Unknown CLI argument	Unrecognized command line argument passed	Warning printed to stderr, argument ignored, program continues	Solved
No upcoming birthdays	No residents match the 14-day window	Explicit message printed to terminal and output file	Solved

Table 3: Error and special situation handling

4 Summary

I started by analysing the task and designing the program structure. Then I implemented file reading, ID parsing and birthday filtering. Finally, I added milestone detection and formatted output.

The most difficult part was the extra functionality, especially checking birthdays within the 14-day range and detecting milestone ages.

The program implementation took about 3 hours, and writing the report took about 4 hours.

Yes, I would recommend this task. It is a good exercise for working with files, strings and basic program structure.

The task was interesting because it uses real Estonian ID codes and practical data processing.

References

[1] Anthropic, "Claude (claude-sonnet-4-6)," Anthropic PBC, San Francisco, CA, USA. Accessed: Mar. 09, 2026. [Online]. Available: https://claude.ai

[2] OpenAI, "ChatGPT (GPT 5.4)," OpenAI, San Francisco, CA, USA. Accessed: Mar. 09, 2026. [Online]. Available: https://chatgpt.com

5 Appendix 1 - extra tast

The program supports comprehensive command line argument handling and a configuration file for default settings.

5.1 Command Line Arguments

The following options can be combined freely in any order:

Argument	**Description**
`-i <filename>`	Override input file path
`-o <filename>`	Override output file path
`--no-terminal`	Disable printing to terminal
`--no-file`	Disable writing to output file

Table 4: Supported command line arguments

5.2 Configuration File

Default settings are stored in `config.conf` as `key=value` pairs. The file is order-independent — each line is parsed individually by matching the key name. If the file is missing, hard-coded defaults are used and a warning is printed to stderr.

```
input_file=Input.txt
output_file=output.txt
print_to_terminal=1
write_to_file=1
```

5.3 Settings Struct

All settings are stored in a global `ProgramSettings` struct loaded at startup. Command line arguments override individual fields without modifying the configuration file.

6 Appendix - 2 screenshots

This section presents several examples of program execution under different conditions. The screenshots illustrate the normal operation of the program as well as several special situations.

6.1 Normal program execution

Figure 1 shows the typical output of the program when valid input data is provided and matching birthdays are found.

```
PS C:\Users\giley\CLionProjects\BigHomework1WithExtra> gcc *.c -o run.exe
PS C:\Users\giley\CLionProjects\BigHomework1WithExtra> ./run.exe
Usage:
  -i <filename>   Set input file
  -o <filename>   Set output file
  --no-terminal   Disable terminal output
  --no-file       Disable file output
Enter date (dd.mm.yyyy): 12.02.2026
Mart Tamm - 12.02.2004 (turns 22)  <-- TODAY!
Jaan Saar - 18.02.1987 (turns 39)
Liis M├ñgi - 24.02.2002 (turns 24)
Karl Lepp - 20.02.1992 (turns 34)
Mari Oja - 14.02.2003 (turns 23)
Grete K╠╜tt - 22.02.2005 (turns 21)
Evelin R├┤ivas - 16.02.1989 (turns 37)
Anneli Vaher - 25.02.2004 (turns 22)
Signe Laas - 23.02.1991 (turns 35)
PS C:\Users\giley\CLionProjects\BigHomework1WithExtra>
```

Figure 1: Normal program execution

6.2 No upcoming birthdays

Figure 2 shows the situation where no residents have a birthday within the next 14 days.

```
PS C:\Users\giley\CLionProjects\BigHomework1WithExtra> ./run.exe --no-file
Usage:
  -i <filename>   Set input file
  -o <filename>   Set output file
  --no-terminal   Disable terminal output
  --no-file       Disable file output
File output disabled.
Enter date (dd.mm.yyyy): 31.08.2008
No upcoming birthdays in the next 14 days.
PS C:\Users\giley\CLionProjects\BigHomework1WithExtra>
```

Figure 2: Program output when no birthdays are found

6.3 Manual input file selection

Figure 3 illustrates the case when the input file name is specified manually by the user.

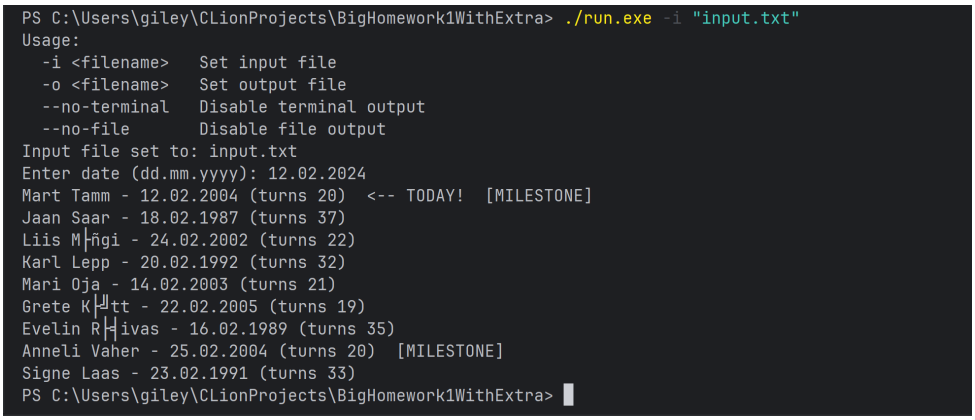


Figure 3: Manual input file selection

6.4 No writing to output file

Figure 4 shows the error message displayed when the input file cannot be found.

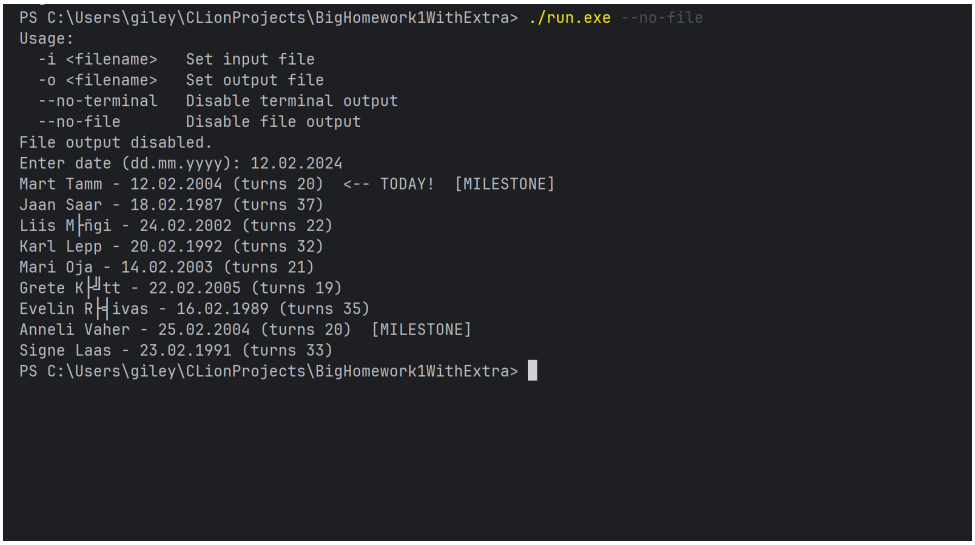


Figure 4: Message when I choosed not to write to output file